itlab-triage and the triage-ops project.
For labeling, enforcing SLOs, and managing workflow labeling there are common policies that are set uniformly for projects in Infrastructure, Platform.
For more information and how to add additional policies see the project README.md.

Retrospectives

At the end of the quarter, or the completion of a large deliverable, teams should perform a retrospective to capture learnings.
There is no set format for the retrospective though Engineering Managers should be aware of the GitLab Retrospective Guidelines.
The retrospective DRI identifies a list of actions which they consolidate in the Summary of Actions section in the issue description.

Process to identify actions:

1. Add a comment on each thread with Actions as the title of the comment (H3 level)
1. Some threads may not require an action, you may want to state this explicitly at the end of the thread for transparency
1. Below the Actions comment, add a suggestion on an action the team can take
1. Ping the contributors to get a round of validation on the actions and potential refinement
1. Create an issue for each action and list them in the Summary of Actions section

---
SECTION: engineering/infrastructure/platforms/tools/_index.md

Tools

The Platforms section builds and maintains various tools to help deploy, operate and monitor our SaaS platforms. The below table is an index to help with the discovery and organization of tools that are actively maintained:

| Tool
| Description |
|---
---|---|
| Tamland | Capacity planning forecasts for GitLab.com |
| Stage Group Ownership Index | Index of stage groups and their owned objects |
| Stage Group Error Budgets | Objective metrics to determine the reliability of a service |
| Service Maturity Model | Overview of each service's operating capabilities |
| Runway | GitLab's internal Platform as a Service implementation |

SECTION: engineering/infrastructure/platforms/tools/runway.md

What is Runway?

Runway is GitLab's internal Platform as a Service implementation, which aims to enable teams to deploy and run their services quickly and safely. Having built-in capabilities for monitoring and scaling means that Development teams can focus on delivering and enhancing their features.

Runway is maintained by the Runway team.

Initial Goals

- Enable Development team to deploy their service using the built-in capabilities for infrastructure, scaling, monitoring that Runway provides.
- Focused on satellite services that are stateless and thus can be autoscaled to meet demand.
- Integration with GitLab's existing tooling (e.g. Pipeline) to enable a streamlined experience.

Services deployed on Runway

- AI-gateway
- Duo Workflow
- PVS Service
- Woodhouse
- glgo
- Runway Docs
- Topology Service
- Contributors Platform

Milestones

Documentation

Runway Documentation

Is Runway suitable for my service?

Yes! Runway is a new initiative and will therefore have some limitations. However, it's important that the team can understand your specific requirements so we can build a robust system that is able to deploy any service.

How do I deploy a service through Runway?

Please reach out to the team following the links below.

How do I get further help?

If you need further support in understanding whether Runway can help you, or you have feedback or questions, please reach out to the Scalability:Frameworks team by either creating an Issue or sending a message in frunway.

Technical Specification

- Epic: Runway - Platform tooling to support AI Innovation
- Runway Docs: Architecture
- Blueprint: GitLab Service-Integration: AI and Beyond

Resources

- Slack channel: frunway
- Issue tracker
- Project repository
- Youtube: Runway Demos

SECTION: engineering/infrastructure/production/_index.md

{{% alert color="warning" %}}
If you're a GitLab team member and are looking to alert Reliability Engineering about an availability issue with GitLab.com, please find quick instructions to report an incident here: Reporting an Incident.
{{% /alert %}}

{{% alert color="warning" %}}
If you're a GitLab team member looking for help with a security problem, please see the Engaging the Security On-Call section.
{{% /alert %}}

The Production Environment

The GitLab.com production environment is comprised of services that operate·or support the operation of·gitlab.com.
For a complete list of production services see the service catalog

How to Get Help

See how to get assistance.

Why infrastructure and production queues?

Premise

Long term, additional teams will perform work on the production environment:

- Release Engineering performs deployments on production
- Security performs scans against the production
- Google may perform work on the underlying production infrastructure

We cannot keep track of events in production across a growing number of functional queues.

Furthermore, said teams will start to have on-call rotations for both their function (e.g., security) and their services. For people on-call, having a centralized tracking point to keep

track of said events is more effective than perusing various queues. Timely information (in terms of when an event is happening and how long it takes for an on-call person to understand what's happening) about the production environment is critical. The production queue centralizes production event information.

Implementation

Functional queues track team workloads (infrastructure, security, etc) and are the source of the work that has to get done. Some of this work clearly impacts production (build and deploy new storage nodes); some of it will not (develop a tool to do x, y, z) until it is deployed to production.

The production queue tracks events in production, namely:

- changes
- incidents
- deltas (exceptions) -- still need to do handbook write up

Over time, we will implement hooks into our automation to automagically inject change audit data into the production queue.

This also leads to a single source of data. Today, for instance, incident reports for the week get transcribed to both the On-call Handoff and Infra Call documents (we also show exceptions in the latter). These meetings serve different purposes but have overlapping data. The input for this data should be queries against the production queue versus the manual build in documents.

Additionally, we need to keep track of error budgets, which should also be derived from the production queue.

We will also be collapsing the database queue into the infrastructure queue. The database is a special piece of the infrastructure for sure, but so are the storage nodes, for example.

For the on-call SRE, every event that pages (where an event may be a group of related pages) should have an issue created for it in the production queue. Per the severity definitions, if there is at least visible impact (functional inconvenience to users), then it is by definition an incident, and the Incident template should be used for the issue. This is likely to be the majority of pager events; exceptions are typically obvious, i.e. they impact only us and customers won't even be aware, or they're alerts that are pre-incident level which by acting on we avoid incidents.

Security Related Changes

All direct or indirect changes to authentication and authorization mechanisms used by GitLab Inc. by customers or employees require additional review and approval by a member of at least one of following teams:

- production team member
- security team member
- developer from a different team that is staff level or higher

This process is enforced for the following repositories where the approval is mandatory using MR approvals:

- gitlab-oauth2-proxy
- gitlabusers

Additional repositories may also require this approval and can be evaluated on a case-by-case basis.

When should we loop the security team in on changes? If we are making major changes to any of the following areas:

1. Processing credentials/tokens
1. Storing credentials/tokens
1. Logic for privilege escalation
1. Authorization logic
1. User/account access controls
1. Authentication mechanisms
1. Abuse-related activities

Type Labels

Type labels are very important. They define what kind of issue this is. Every issue should have one or more.

Label	Description
~Change	Represents a Change on infrastructure please check details on : Change
~Incident	Represents a Incident on infrastructure please check details on : Incident
~Database	Label for problems related to database
~Security	Label for problems related to security

Services

The services list is mentioned here : https://gitlab.com/gitlab-com/runbooks/blob/master/services/service-catalog.yml

Always Help Others

We should never stop helping and unblocking team members. To this end, data should always be gathered to assist in highlighting areas for automation and the creation of self-service processes. Creating an issue from the request with the proper labels is the first step. The default should be that the person requesting help makes the issue; but we can help with that step too if needed.

If this issue is urgent for whatever reason, we should label them following the instructions above and add them to the ongoing Milestone.

On-Call Support

For details about managing schedules, workflows, and documentation, see the on-call documentation.

On-Call escalation

Given the number of systems and service that we use, it's very hard if not impossible to reach an expert level in all of them. What makes it even harder is the rate of changes made to our infrastructure. For this reason, the person on-call is not expected to know everything about all of our systems. In addition, incidents are often complex and vague in their nature requiring different perspectives and ideas for solutions.

Reaching out for help is considered good practice and should not be mistaken for incompetence. Asking for help while following the escalation guidelines and checklists can expose information and result in faster resolution of problems. It also improves the knowledge of the team as a whole when for example an undocumented problem is covered in runbooks after an incident or when questions are asked in Slack channels where others can read it. This is true for on-call emergencies as well as project work. You will not be judged on the questions you ask, regardless of how elemental they might be.

The SRE team's primary responsibility is availability of gitlab.com. For this reason, helping the person on-call should take priority over project work. It doesn't mean that for every single incident, the entire SRE team should drop everything and get involved. However, it does mean that as knowledge and experience in a field that is relevant to a problem, they should feel entitled to prioritize that over project work. Previous experiences have shown that as the incident's severity increased or potential causes were ruled out, more and more people from across the company were getting involved.

Production Events Logging

All configuration, deploys and feature flag events are recorded in Elastic Search using the events index.
To access events and for more information about how they are logged see the events runbook.

These events include deploys, feature-flags, configuration (Chef, Terraform), and all Kubernetes changes.
Events are recorded separately for the staging and production environment.

Incident Subtype - Abuse

For some incidents, we may figure out that the usage patterns that led to the issues were abuse. There is a process for how we define and handle abuse.

1. The definition of abuse can be found on the security abuse operations section of the handbook
1. In the event of an incident affecting GitLab.com availability, the SRE team may take actions

immediately to keep the system available. However, the team must also immediately involve our security abuse team. A new security on call rotation has been established in PagerDuty - There is a Security Responder rotation which can be alerted along with a Security Manager rotation.

Backup and Restore

See policies for Backup and Restore.

Patching

Policy

All servers in the production environment managed and maintained by the GitLab infrustructure team will be proactively maintained and patched with the latest security patches.

Summary of Patching Strategy

All production servers managed by chef have a base role that configures each server to install and use unattended-upgrades for automatically installing important security packages from the configured apt sources.
Unattended-upgrades check for updates everyday between 6 and 7 am UTC. The time is randomized to avoid hitting the mirrors at the same time. All output is logged
to /var/log/unattended-upgrades/.log.

Unattended upgrades is configured to automatically patch all security upgrades for packages with the exception of the GitLab omnibus package.

The critical change process is described in the emergency change process overview.

Patching Validation

Patch validation can be performed in 3 ways.

- Manually by cross examining the logs of the host with the vulnerability finding in wiz.io.
- Reviewing vulnerability & tracking issue raised into GitLab by [Vulnerability Management teams automation] (/handbook/security/product-security/vulnerability-management/automation/)
- Reach out to Vulnerability Management in slack gvulnerabilitymanagement

General OS (Ubuntu or other Linux) Version updates

Infrastructure will look to begin OS upgrades for Ubuntu LTS releases 6 months after their release and attempt to maintain all GCP compute instances on an LTS within the last 5 years of release. We leverage Ubuntu Pro to gain an extension on security updates for older OS releases using their ESM service, and Ubuntu Livepatch to automatically apply Kernel security updates to running systems.

Penetration Testing

Infrastructure will provide support to the security team for issues found during
penetration testing. For coordinating a pen test or to coordinate any procedures to address and

remidiate
vulnerabilities, should be communicated to the infrastructure team through an issue in the infrastructure issue tracker.

In the issue please provide the following:

- scope of the testing
- a suggested time frame
- the depth of testing
- which services will be tested
- the procedures being done
- any possible teams that may be affected (such as support, security, etc)

Please tag issues with the ~security label and /cc infrastructure managers.

SECTION: engineering/infrastructure/production/readiness.md

Overview

The Production Readiness Review is a process that helps identify the reliability needs of a service, feature, or significant change to infrastructure for GitLab.com.
It loosely follows the production readiness review from the SRE book.

The goal of the readiness review is to make sure we have enough documentation, observability, and reliability for the feature, change, or service to run at GitLab.com production scale.
The readiness review process should be started as early as possible as features progress through our product maturity levels.

Completing a readiness review doesn't necessarily mean that the Infrastructure teams will take over on-call responsibilities or ownership from the service team. If required, this should be discussed in the merge request.

This review is meant to facilitate collaboration between Service Owners, Security and Infrastructure teams to help bridge any gaps identified for a new service.
The review document will serve as a snapshot of what is being deployed and the discussions that surround it.
It is not intended to be constantly updated.

The readiness review MR will go through a single review for every maturity level.
We require an MR because it allows for inline comments, threaded discussions and explicit approval.
Once an MR has been approved by the stakeholders and merged it is considered approved for corresponding level.

The readiness review issue is used to coordinate among stakeholders who will be assigned as reviewers and to track progress using the issue boards below:

| Planning | Status|

|--------------|-------------|
Readiness Planning Board	Readiness Status Board
Readiness currently being prepared.	Readiness actively in review.

Criteria for starting a Production Readiness Review

Production Readiness should start as early as possible and is required for all product maturity levels that meet any of the following criteria:

- New infrastructure components, or significant changes to existing components that have dependencies on the GitLab application.
- Changes to our application architecture that change how the infrastructure scales, or how data is processed or stored.
- New services or changes to existing services that will factor into the availability of the GitLab application.

If none of the above criteria is met, it will be unlikely that a Production Readiness review is necessary.

Process

The Production Readiness process is authored by the DRI of the work that is being delivered.

1. Create an issue using the issue template in the readiness project. The title of the issue should be a descriptive name of change.
2. Follow the Readiness Checklist in the template.

The issue template will guide you though preparing your merge request and how to use the approriate labels to keep your review moving through the process.

The template also contains information about what is expected for Experimental, Beta, and Generally Available features and services.

Guidelines for the author

- If this is a review for the next maturity level for an existing feature, create an MR that modifies the existing review document.
- Do not remove any items from the template even if they are not applicable to the feature.
- The content of the readiness review should primarily consist of links to other docs and/or short explanations about why or why not we are meeting requirements.
  It is not meant to contain detailed architecture, diagrams, or explanations on design. By linking to these documents we retain a single source of truth for this information rather than have it duplicated in the Readiness Review MR.
  Design and architecture proposals should use the Architecture Design Workflow process.
- Be as descriptive as possible when writing this review. Avoid terminology that makes it appear as if something is already well known.
  What may be known by one may not be well understood by another.
- Make no assumptions. If an answer cannot be provided, it is better to explain why we lack the ability to provide details.

This assists in fostering discussion and enables us to spawn new issues if we identify areas of improvement.
This is also an excellent avenue for asking for help as needed.
- It is acceptable to have issues created for line items which may not be complete or not yet well known.
Link to those issues as necessary. One issue must NOT be created as a catchall;
instead for each item, a dedicated issue should be created. As details emerge from those issues, that information should then be fed back into the readiness review.
- It is also acceptable to have issues created as a basis for visiting after the service has been introduced into production.
For these items, an issue must already be created and a statement indicating why it is safe to proceed without it being a blocker.
- For any question or section which simply may not have an answer, or if it isn't relevant, leave it in the MR and state why it is not applicable.
Doing so ensures that we've performed a review of all possible subject areas.

Guidelines for the reviewer

- As a reviewer of the Production Readiness proposal,
your task is to collaborate with the author and decide whether information provided in the proposal is sufficient for production readiness.
Ultimately the author is the DRI and responsible for putting the service in production.
This review helps the DRI work with you to ensure that what is being proposed meet our reliability needs.
- Consider how sections listed in the issue template are addressed.
It is important that you highlight any shortcomings that you observe.
Ensure that non-applicable sections are properly noted and that issues are created if there are gaps that need to be addressed following the change.
- Leave questions as you would with regular code review.

Completing the readiness review

Once all discussions have been addressed all mandatory items have satisfactory answers, the author will request approvals from the reviewers.
The reviewers should note their approval by approving the merge request.
Following this, the issue will be closed and the change can be applied in production.

SECTION: engineering/infrastructure/production/architecture/_index.md

Our GitLab.com core infrastructure is primarily hosted in Google Cloud Platform's (GCP) us-east1 region (see Regions and Zones).

This document does not cover servers that are not integral to the public facing operations of GitLab.com.

Purpose

This page is our document that captures an overview of the production architecture for

GitLab.com.

Scope

The compute and network layout that runs GitLab.com

Roles and Responsibilities

Role	Responsibility
Infrastructure Team	Responsible for configuration and management
Infrastructure Management (Code Owners)	Responsible for approving significant changes and exceptions to this procedure

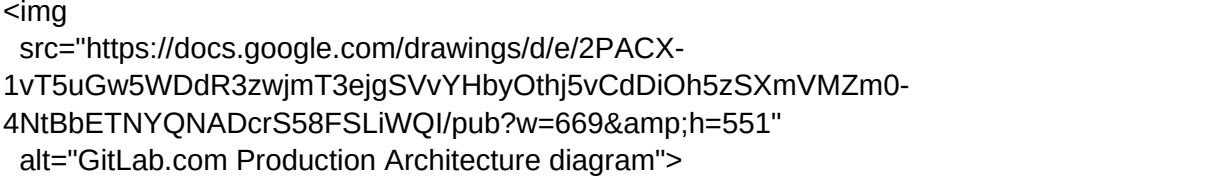
Procedure

Related Pages

- Application Architecture documentation
- GitLab.com Settings
- GitLab.com Rate Limits
- Monitoring of GitLab.com
- GitLab performance monitoring documentation
- Performance of the Application
- Gemnasium Service Production Architecture
- CI Service Architecture
- dev.gitlab.org Architecture
- ops.gitlab.net Architecture
- version.gitlab.com Architecture

Current Architecture {infra-current-archi-diagram}

GitLab.com Production Architecture {gitlab-com-architecture}



Source, GitLab internal use only.

Most of GitLab.com is deployed on Kubernetes using GitLab cloud native helm chart. There are a few exceptions for this
which are mainly the datastore services like PostgresSQL, Gitaly, Redis, Elasticsearch.

Cluster Configuration {cluster-configuration }

GitLab.com uses 4 Kubernetes clusters for production with similarly configured clusters for staging.

One cluster is a Regional cluster in the us-east1 region, and the remaining three are zonal clusters that correspond to GCP availability zones us-east1-b, us-east1-c and us-east1-d. The reasons for having multiple clusters are as follows:

- Ensure that high-bandwidth services do not send network traffic across zones.
- Isolation of workloads
- Isolation of maintenance changes and upgrades to the clusters

For more information on why we chose to split traffic into multiple zonal clusters see this issue exploring alternatives to the single regional cluster.
A single regional cluster is also used for services like Sidekiq and Kas that do not have a high bandwidth requirement and services that are a better fit for a regional deployment.

In keeping with GitLab's value of transparency, all of the Kubernetes cluster configuration for GitLab.com is public, including infrastructure and configuration.

The following projects are used to manage the installation:

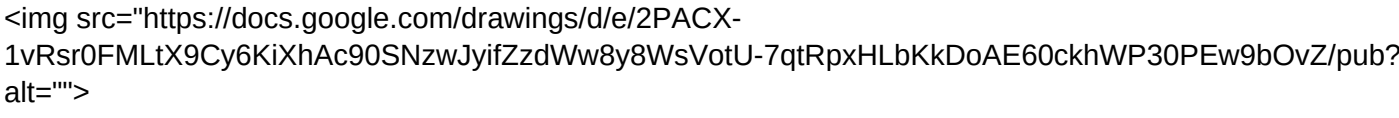
- k8s-workloads/gitlab-com: Contains the GitLab.com configuration for the GitLab helm chart.
- k8s-workloads/gitlab-helmfiles: Contains the configuration cluster logging, monitoring and integrations like PlantUML.
- k8s-workloads/tanka-deployments: Contains the configuration for Jaeger and other services not directly related to the GitLab application.
- config-mgmt: Terraform configuration for the cluster, all resources necessary to run the cluster are configured here including the cluster, node pools, service accounts and IP address reservations.
- charts: Charts created by the infrastructure department to deploy services that don't have community charts.

All inbound web, git http, and git ssh requests are received at Cloudflare which has HAProxy as an origin. For Sidekiq, multiple pods are configured for Sidekiq cluster to divide Sidekiq queues into different resource groups. See the chart documention for Sidekiq for more details.

Monitoring and Logging

Monitoring for GitLab.com runs in the same cluster as the application. Metrics are aggregated in the ops cluster using Mimir, which we interface with via Grafana that has multiple components.

Prometheus is configured using the kube-prometheus-stack helm chart in the namespace monitoring, and every cluster has its own Prometheus which gives us some sharding for metrics.



Alerting for the cluster uses generated rules that feed up to our overall SLA for the platform.

Logging is configured using tanka where the logs for every pod is forwarded to a unique Elasticsearch index. fluentd is deployed in the namespace logging.

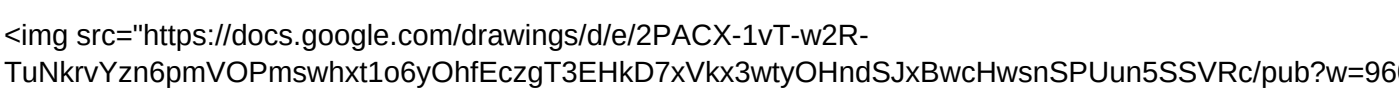
Cluster Configuration Updates

There is a single namespace gitlab that is used exclusively for the GitLab application.
Chart configuration updates are set in the gitlab-com k8s-workloads project where there are yaml configuration files that set defaults for the GitLab.com environment with per-environment overrides.
Changes to this configuration are applied by the SRE and Delivery team after a review using a MR review workflow.
When a change is approved on GitLab.com the pipeline that applies the change is run on a separate operations environment to ensure that configuration updates do not depend on the availability of the production environment.

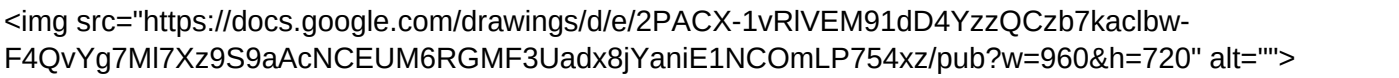
For namespaces in the cluster for other services like logging, monitoring, etc. a similar GitOps workflow is followed using the gitlab-helmfiles and tanka-deployments.

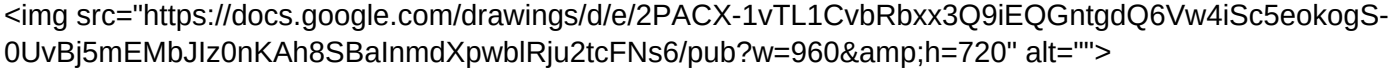
GitLab.com does not depend on itself when pulling images utilized in our Kubernetes clusters. Instead, we utilize our dev.gitlab.org container registry for CNG images.
This is to ensure that during an incident, we will still maintain the ability to pull images and run our applications as necessary.
For any image that we do not build ourselves, these may be pulled from Docker Hub. Conveniently, these images are mirrored on Google's Container Registry product.
Our GKE nodes are configured from the start with this mirror already in place providing further redundancy in the event that the Docker Hub is unavailable.

Database Architecture



Redis Architecture





GitLab.com uses several Redis shards for various use cases such as caching, rate-limiting, Sidekiq queueing. More info on various Redis shards, their configuration, and usage can be found in the chef-repo and GitLab. The relationship between

Redis instances and GitLab deployments can be tracked via this Grafana link.

When needed we also sometimes deal with CPU saturation by making application changes. Some of the techniques for this are discussed in this video.

Network Architecture

Source, GitLab internal use only

Our network infrastructure consists of networks for each class of server as defined in the Current Architecture diagram. Each network contains a similar ruleset as defined above.

We currently peer our ops network. Inside of this network is most of our monitoring infrastructure where we allow InfluxDB and Prometheus data to flow in order to populate our metrics systems.

For alert management, we peer all of our networks together such that we have a cluster of alert managers to ensure we get alerts out no matter a potential failure of an environment.

No application or customer data flows through these network peers.

DNS & WAF

GitLab leverages Cloudflare's Web Application Firewall (WAF). We host our Domain Name Service (DNS) with Cloudflare (gitlab.com, gitlab.net) and Amazon Route 53 (gitlab.io and others). For more information about CloudFlare see the runbook and the architecture overview.

TLD Zones

When it comes to DNS names all services providing GitLab as a service shall be in the gitlab.com domain, ancillary services in the support of GitLab (i.e. Chef, ChatOps, VPN, Logging, Monitoring) shall be in the gitlab.net domain.

Remote Access

Access is granted to only those whom need access to production through bastion hosts. Instructions for configuring access through bastions are found in the bastion runbook.

Secrets Management

GitLab utilizes two different secret management approaches, GKMS for Google Cloud Platform (GCP) services, and Chef Encrypted Data Bags for all other host secrets.

GKMS Secrets

For more information about secret management see the runbook for Chef secrets using GKMS, Chef vault and how we manage secrets in Kubernetes.

Monitoring

See how it's doing, for more information on that, visit the monitoring handbook.

Exceptions

Exceptions to this architecture policy and design will be tracked in the compliance issue tracker.

References

- Controlled Document Procedure

SECTION: engineering/infrastructure/production/architecture/ci-architecture.md

This document only covers our shared and GitLab shared runners, which are available for GitLab.com users and managed by the Infrastructure teams.

General Architecture {ci-general-arch}

Our CI infrastructure is hosted on Google Cloud Engine (GCE). In GCE we use the us-east1-c and us-east1-d All of them are configured via Chef. These machines are manually created and added to chef and do NOT use terraform at this time.

In each region we have few types of machines:

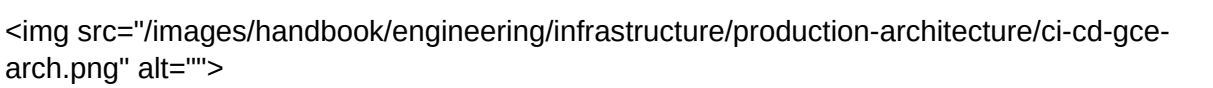
- X-runners-manager - hosts with the GitLab Runner process. These hosts handle job's execution and autoscaled machines scheduling. For the second task they are using Docker Machine and API exposed by used cloud provider.
- autoscaled-machine-N - hosts created and managed by the GitLab Runner. They are used to run a job's scripts inside of Docker containers. Currently we're allowing a machine to be used only by one job at once. Machines created by gitlab-shared-runners-manager-X and private-runners-manager-X are re-used. However, machines created by shared-runners-manager-X are removed immediately after the job is finished.
- Helpers - hosts that provide auxiliary services such as monitoring and cache.
  - Prometheus - Prometheus servers in each region monitor machines.

Runner managers connect to the GitLab.com and dev.gitlab.org API in order to fetch jobs that need to be run. The autoscaled machines clone the relevant project via HTTPS.

The runners are connected as follows:

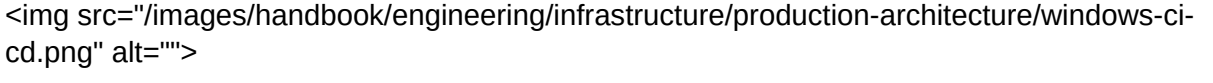
- shared-runners-manager-X (srm): connected to GitLab.com and are available to all GitLab.com users as shared runners. Privileged mode is off.
- gitlab-shared-runners-manager-X (gsrm): connected to GitLab.com and dev.gitlab.org. They are available as shared runners to GitLab.com and tagged with gitlab-org. They provide services to gitlab-ce and gitlab-ee projects and all their forks. They are the generic shared runners on dev.gitlab.org. Privileged mode is off.
- private-runners-manager-X (prm): connected to both GitLab.com and dev.gitlab.org. They are registered as specific runners to the GitLab application projects and used only by us. Privileged mode is on.

Detailed Architecture {ci-detailed-arch-diagram}



Source

Windows Architecture



Source

Data Flow {ci-data-flow}

Management Data Flow

- prometheus.gprd.gitlab.net scrapes each runner host with the job ci-node. Prometheus also scrapes specific prometheus nodes within the runners' regions using prometheus federation.
- chef.gitlab.com server is accessed by all hosts from inside of Cloud Provider Region, excluding autoscaled machines.

GitLab Data Flow

- runners-manager-X hosts are connected to one or more GitLab instances and are constantly asking the API for new jobs that should be executed. After the job is started Runners are also updating the job's trace and status by sending updates to the GitLab instance. This communication uses Runner's API from GitLab APIv4.

- autoscaled-machine-N hosts first access GitLab with the git+http(s) protocol to receive project sources with git pull or git fetch operations, depending on configuration. This operation uses the general git+http(s) protocol and specific type of authentication (using gitlab-ci-token feature). The job may also access project's submodules using GitLab with the same protocol as for the project. These hosts may also upload and/or download artifacts to and from GitLab. The gitlab-runner-helper binary is used for

this purpose which uses Runner's API from GitLab APIv4.

Cloud Region Internal Communication

- runners-manager-X to autoscaled-machine-N - Runner starts jobs on autoscaled machines using the Docker Engine API. After the machine is created, Runner receives IP:PORT information about where the Docker Engine API endpoint is available. In GCE this uses the internal IP address. Using the Docker Engine API, Runner first schedules the different containers used for the purpose of the job. It then starts job's scripts and receive commands output. This output is then sent upstream to GitLab as it was described above.

- prometheus-X to autoscaled-machine-N - the Prometheus server requests the autoscaled machines for exported metrics. It uses the HTTP(S) protocol for this.

Cloud Region External Communication

- runner-manager-X to Cloud Provider API Gateway - Runner, using Docker Machine, manages autoscaled machines used for executing jobs. It uses Cloud Providers API to schedule machines creation and removals. It also uses the same API to gather information about existing machines.

- After creating the machine Runner uses received IP:PORT to schedule containers and execute jobs scripts there.

- autoscaled-machine-N to external Docker Registry - Docker Engine, using Docker Registry API, pulls Docker Images from external machines. This could be Docker Hub, GitLab Registry, or any other Docker compatible registry.

Deployment and Configuration Updates {ci-configuration}

The Runner and it's configuration is handled with Chef and defined on chef.gitlab.com. The detailed upgrade process is described in the associated runbook.

In summary:

- For a configuration update we need to:
  - update definitions in chef-repo and upload new definitions to chef.gitlab.com,
  - execute sudo chef-client on nodes where needed.
- For a Runner version update:
  - update definitions in chef-repo and upload new definitions to chef.gitlab.com,
  - execute sudo /root/runnerupgrade.sh on nodes where needed.

Why the difference?

When we're updating Runner, the process needs to be stopped. If this is done

during job's execution, it will break the job. That's why we use Runner's feature named graceful shutdown. By sending SIGQUIT signal to the Runner, we're causing Runner to not request new jobs but still wait for existing ones to finish. If this was done from inside of chef-client run it could fail in unexpected way. With the /root/runnerupgrade.sh script we're first stopping Runner gracefully (with 7200 minutes timeout) and then starting chef-client to update the version.

For Runner's configuration update there is no need to stop/restart Runner's process and since we're not changing Runner's version, chef-client is not upgrading package (which could trigger Runner's process stop). In that case we can simply run sudo chef-client. This will update the config.toml file and Runner will automatically update most of the configuration.

Some of the general configuration parameters can't be refreshed without restarting the process. In that case we need to use the same script as for the Runner Upgrade.

Important Links and Metrics {ci-important-info-links}

Monitoring Information

- prometheus-app.gprd.gitlab.net - for metrics scraped from GitLab via unicorn exporter and GitLab Monitor project
- prometheus.gprd.gitlab.net - for Runner internal metrics, node metrics of Runner Manager machines, gathering metrics about our cloud providers, gathering metrics of autoscaled machines with federation from CI Prometheus servers (Ben is currently working on enabling Thanos there, so Grafana will access CI Prometheus servers directly)
- prometheus-01.nyc1.do.gitlab-runners.gitlab.net, prometheus-01.us-east1-c.gce.gitlab-runners.gitlab.net, prometheus-01.us-east1-d.gce.gitlab-runners.gitlab.net - for scraping metrics from exporters installed on autoscaled machines - currently node exporter only.
  - In GCP we're using native GCP service discovery support that is available in Prometheus.
- Alerts are sent to ci-cd-alerts channel on Slack

Monitoring Links

- CI Monitoring Overview
- CI Cloud Provider Stats
- CI Autoscaling Stats
- CI Logs (only GCP)
- CI Shared Runners Sentry (srm machines)
- CI Internal Runners Sentry (gsrm & prm machines)
- CI Alert Configuration

SECTION: engineering/infrastructure/production/architecture/disaster-recovery.md

This page has moved to a single internal handbook page

SECTION: engineering/infrastructure/production/architecture/supporting-architecture.md

This document covers architectures that support GitLab.com functions, but are not user facing and are managed by the Infrastructure teams.

dev.gitlab.org {dev-gitlab-org}

Dev.gitlab.org is a GitLab instance hosted in Azure. The instance is running a vanilla GitLab Community Edition package, from a nightly build built from main branch of all GitLab components. The instance is automatically upgraded daily using the cron defined in the gitlab-server cookbook with a role override set in the chef-repo role (GitLab internal only).

It's primary use is for building official Docker images and GitLab packages which are later used as part of the official release pipelines.

It is also used as an OAuth authentication service which allows users to sign in to other services using their dev.gitlab.org account, such as:

Sentry
Version app

Source, GitLab internal use only

Architecture

Dev.gitlab.org runs on a single VM, and is using the official Linux package bundled database, Redis and other services. The repositories are stored on a dedicated SSD, while artifacts, LFS objects, Container Registry objects and uploads are stored in GCS.

Database backups, and repository backups are automatically created using the built-in package backup procedure that runs prior to the package upgrade.
These backups are automatically uploaded to an AWS S3 bucket configured in the specific chef role using the official Linux package auto-backup feature.

Source, GitLab internal use only

ops.gitlab.net {ops-gitlab-net}

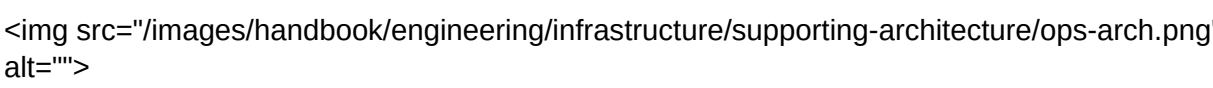
Ops.gitlab.net is a GitLab instance hosted in GCP. The instance is running a vanilla GitLab EE package, from the official release channel. The instance is automatically upgraded using the cron defined in the gitlab-server cookbook with a role override set in the chef-repo role (GitLab internal only).

It's primarily used for operational tasks.

It contains repositories for managing GitLab.com's infrastructure and as a mirror of the infrastructure repositories.
It also hosts various tools for managing deployments and useful chatops commands that are sent from Slack.
Admins for ops.gitlab.net are the Infrastructure Managers - Reliability, Scalability, Delivery.

Architecture

The instance runs on a single VM, and is using CloudSql as a database backend, and Memorystore (managed Redis service). The repositories are stored on a dedicated SSD, while artifacts, LFS objects, Container Registry objects and uploads are stored in GCS.



Source, GitLab internal use only

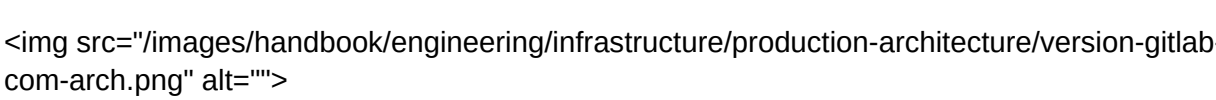
version.gitlab.com

The version service is hosted in Google Cloud on a Kubernetes cluster.
Version is used to store available GitLab versions as well as if it contains a vulnerability, render version check badge for self-managed GitLab instances, and collect data sent during version check
and usage ping from self-managed instances.

Architecture

We are running the version.gitlab.com on Kubernetes and use Auto DevOps for managing deployments. The application runs on multiple pods. Google Cloud SQL with PostgreSQL is used by the pods to store data, Cloud SQL has two replicas configured. Google Cloud Memorystore is used as a cache store.

Production environment



Source, GitLab internal use only

SECTION: engineering/infrastructure/rate-limiting/_index.md

Overview

Rate limiting is a critical feature in GitLab that enhances security and performance by restricting the number of requests users or IP addresses can make within a set timeframe. GitLab's approach to rate limiting helps prevent abuse, ensures fair resource allocation, and maintains system stability, effectively safeguarding against potential attacks and ensuring a reliable user experience.

Rate limited requests will return a 429 - Too Many Requests response.

Processes

- Changes to rate limits require a Change Request.
- Request assistance for a user's rate limiting settings with this issue template.
- For internal teams seeking a bypass, please refer to the Rate Limit Bypass Policy.

GitLab Rate Limit Architecture

GitLab.com

```
mermaid
flowchart TD
    A[Internet]
    A -->|gitlab.com| D(Cloudflare WAF)
    A -->|customers.gitlab.com| F
    subgraph ide3 [Cloudflare WAF]
        D[gitlab.com zone]
        F[customers.gitlab.com zone]
    end
    A -->|.gitlab.io| K
    A -->|registry.gitlab.com| N
    D --> G
    F --> H
    G[https + ssh]
    subgraph ide2 [GitLab]
        subgraph ide1 [HAProxy]
            N[registryhttps]
        end
        H[nginx]
        K[pages kubernetes limits]
        K-->M[pages application limits]
        I[Rack::Attack]
        J[Gitlab::ApplicationRateLimiter]
    end
    G --> I
    G --> J
```

Cloud Connector

```
mermaid
flowchart LR
  A[Internet]
  subgraph cf [Cloudflare WAF]
     E[cloud.gitlab.com zone]
  end
  subgraph G [GitLab]
    subgraph ccbe [CloudRun / GKE]
      F[Cloud Connector backends]
    end

    subgraph GKE
      C[gitlab.com]
    end

    subgraph AWS
      D[GitLab Dedicated]
    end
  end

  E --> F
  A -->|cloud.gitlab.com| E
  C -->|cloud.gitlab.com| E
  D -->|cloud.gitlab.com| E
```

Rate limits exist in multiple layers for GitLab.com, each boxed area above represents a place where rate limits are implemented.

Limits

To minimise duplication, please see the following sources to determine the current active rate limits:

<table>
<tr>
<th>
Cloudflare
</th>
<td>

- GitLab.com:
  - Cloudflare Dashboard
  - Base Cloudflare Rules Terraform shared with Dedicated.
  - GitLab.com Cloudflare Rules Terraform
- Cloud Connector:
  - Cloudflare Dashboard
  - Runbook + TF links

</td>

</tr>

<tr>
<th>
Application
</th>
<td>

- Application Settings (admin access only)
  - See User and IP Rate Limits and Protected Paths
- GitLab.com Docs (published manually)

</td>
</tr>

</table>

Bypasses

Published rate limits apply to all customers and users with no exceptions.

Customers or internal teams seeking a bypass should refer to the Rate Limit Bypass Policy.

Traffic management Rate Limits

Cloudflare

Cloudflare serves as our "outer-most" layer of protection, sitting at the network edge on inbound traffic. We use Cloudflare's standard DDoS (Distributed Denial of Service) protection plus Spectrum to protect git over ssh.

How rate limits are applied in Cloudflare:

<table>
<tr>
<th>
Gitlab.com Page Rules
</th>
<td>

- Configured by Terraform in config-mgmt.
- URL pattern matching.
- Control Cloudflare's DDoS interventions and caching (e.g. bypasses, security levels etc).
- No rate limits configured by default, but they can be enabled when under an attack

</td>
</tr>
<tr>

<th>

Gitlab.com Rate Limiting
</th>
<td>

- Configured by Terraform in config-mgmt with base limits shared with Dedicated and GitLab.com specific ones rules.
- Covers a wide range of cases:
  - Global limits per IP
  - Global limits per session (cookies) or tokens (headers) can be used as rate counters to avoid IP scope false positives, e.g. many users behind a single IP, VPN.
- Endpoint-specific limits.
- Independent of application rate limits.

</td>
</tr>

<th>
Cloud Connector Rate Limiting
</th>
<td>

- Configured by Terraform (see runbook links).
- Throttles requests from both end-user clients such as IDEs as well as GitLab Rails instances (GL.com, SM and Dedicated.)
- Limits are counted against any GitLab user's anonymous global user ID, regardless of where the request originates from.
- Primarily used to throttle consumption of non-horizontally scalable resources such as AI vendor limits.
- Can be configured for each Cloud Connector backend individually.

</td>
</tr>

</table>

Note: Cloudflare is not application aware and does not know how to map to our users and groups.

Cloudflare is also responsible for applying the X-GitLab-Rate-Limit-Bypass header to a subset of requests from:

- Legacy customer IP rate limit bypasses
- Third party vendors with whom we have integrations
- Internal infrastructure

For a full list of conditions where the header will be applied, see this configuration.

Our Cloudflare runbook contains more detail on configuring this layer of our infrastructure.

Changes to Cloudflare rate limits require a Change Request, and should
be discussed with the Production Engineering::Foundations SRE team before implementing.

HAProxy

The majority of GitLab's traffic management rate limits have been moved out of HAProxy and into Cloudflare (see this confidential issue for more details).

However, HAProxy is still responsible for Registry rate limits, as that component is not fronted by Cloudflare. See Registry for more details.

Currently HAProxy also handles applying the X-GitLab-Rate-Limit-Bypass for a limited number of special paths. The list can be found here, but there is work underway to move this logic out into Cloudflare as well.

Application Rate Limits

There are multiple application rate limiting mechanisms in place that can often be referred to interchangeably. These allow for more informed traffic management compared to that implemented by Cloudflare as the application has access to user information:

- Rack Attack
- Application Rate Limiter
- Plan Limits

Please see Application Limits Development to contribute application limits to GitLab.

Overview

The rate limit period is 1 minute (60 seconds).

<table>
<tr>
<th>

Category

</th>
<th>

Identifier

</th>
</tr>

<tr>
<th>

Unauthenticated

</th>
<td>

IP address

</td>
</tr>

<tr>
<th>

Authenticated

</th>
<td>

User information (user, project, etc)

</td>
</tr>

<tr>
<th>

Protected Paths

</th>
<td>

Configurable list of paths e.g. /user/signin

</td>
</tr>

</table>

RackAttack

GitLab utilises RackAttack as middleware to throttle Rack requests. These can be configured by extending Gitlab::RackAttack and Gitlab::RackAttack::Request.

For more information about configuring rate limits for a GitLab instance, see the User and IP rate limits doc.

You can read more information about rate limits specific to GitLab.com, alongside RackAttack configuration documentation in runbooks.

ApplicationRateLimiter

The GitLab application has simple rate limit logic that can be used to throttle certain actions which is used when we need more
flexibility than what Rack Attack can provide, since it can throttle at the controller or API level. These rate limits are configured in applicationratelimiter.rb. The scope is up to the

individual limit implementation and can be any ActiveRecord object or combination of multiple. It is commonly per-user or per-project (or both), but it can be anything, e.g. the RawController limits by project and path.

There is no way to bypass these rate limits (e.g. for select users/groups/projects); when the rate limit is reached a plain response with a 429 status code is issued without rate limiting headers.

Instructions for configuring these rate limits can be found in the GitLab docs.

For more information about introducing new Rate Limits for GitLab, see the Product Processes Handbook page.

Other Rate Limits

GitLab Pages

Because GitLab Pages is not behind CloudFlare and doesn't have CDN support, rate limits are set in several places:

1. Within the Kubernetes settings for GitLab.com.
2. In the Go ratelimiter module for Pages.

For more information, see the Pages Rate Limit documentation.

Registry

Registry is not fronted by Cloudflare (see this confidential issue for full details around why). There is no application-level setting for it either (though there is an open feature request to add that), so the only place where there is rate limiting in place for Registry is in HAProxy.

There are no rate limit exceptions available for Registry.

Headers

The list of semi-standard rate limiting response headers can be found here.

- Cloudflare does not return rate limit response headers on any request.
- RackAttack returns rate limit response headers on throttled requests only.
- ApplicationRateLimiter does not return rate limit response headers.
- GraphQL endpoints currently do not return rate limit response headers.

See this issue for improvements to returning rate limiting response headers.

Client-Side Best Practices

To minimize the risk of hitting rate limits, you can try the following:

1. Implement Retry Logic

   - Configure exponential back off and retry for failed attempts.
   - Respect the 429 response status and Retry-After headers.
   - Implement circuit breakers for persistent failures.
1. Stagger Automated Pipelines
   - Reduce the volume of requests being made at any one time.
1. Request Batching
   - Combine multiple operations into single requests where possible.
   - Implement client-side queue management.
1. Implement Caching
   - Cache responses where possible to reduce request frequency.
   - Implement conditional requests, utlizing If-Modified-Since for example.
1. Monitor for Rate Limited Responses
   - Log and alert on unexpected increases in rate limited requests.
   - Track rate limit responses and headers where applicable.

 Troubleshooting

Please see Rate Limiting Troubleshooting.

 Important Links

- docs: GitLab.com
- docs: Self Managed (and Dedicated)
- runbook: GitLab.com rate limiting
- handbook: Identifying the cause of IP Blocks on GitLab.com

SECTION: engineering/infrastructure/rate-limiting/bypass-policy.md

Bypass Policy for GitLab.com

Published rate limits apply to all customers and users with no exceptions. Rate limiting bypasses are only considered in exceptional circumstances for customers experiencing a high severity incident, such as a full outage. Even with this requirement satisfied, it is still possible that a bypass request will be rejected. Permanent bypasses are not made.

The reason for this strict policy is we want to help every customer as much as possible when they run into problems, but on GitLab.com, we have to prioritize the reliability and stability of the entire instance to protect all customers and users. Allowing even one customer to bypass rate limits creates a high risk situation where we have opened an avenue for a high severity production incident (such as GitLab.com being unavailable to all users) to occur. This is why it is always preferred to find ways to resolve the issue without needing to bypass rate limits.

 Bypass Requests

- Bypass requests can only be made for up to a 2 week period. Anything longer will be denied.
- If the 2 week period is ending and a customer needs more time, a new request can be made to extend the bypass, but it is not guaranteed.

We have two levels of rate limiting that we can potentially bypass:

1. Rack Attack

- This is the most common set of rate limits that users hit. It exists at the application level.
- These will be found in the Kibana logs.
- As this is the most common bypass granted, requests may need to be denied if too many IPs are already on the bypass list, so be sure to only request when absolutely necessary to avoid potentially denying another customer that might have a higher severity need.

2. Cloudflare

- This is uncommon for users to hit. It exists at the network edge as a safety net above all the other rate limits to protect against DDoS (Distributed Denial of Service) and other attacks.
- These will be found only in Cloudflare logs. If you are seeing 429 responses and are unable to find matching log records in Kibana, then it is likely you are hitting the Cloudflare rate limits.
- Bypasses are rarely approved at the Cloudflare level due to the risk involved in opening IPs with no limit at this level, so if this is the rate limit being hit, it is likely that the bypass will be denied.

Considerations

Things we consider when granting a bypass:

- How much additional traffic will be allowed through due to the bypass
- How many other IPs are currently being bypassed
- The current stability of GitLab.com
- Time of year (holidays, etc that could effect traffic)

This means that even if a customer checks all the boxes needed to obtain a bypass, there is still a chance that it will be denied due to reasons unrelated to the request.

Requesting a Bypass

Before you request a bypass, please ensure you have:

- Identified why rate limits have started to be hit.
- Have tried to investigate and implement other solutions to mitigate the problem (such as staggering execution of automated pipelines, exponential backoff and retry for failed authentication attempts, etc...).
- Have a plan of action that the customer and/or Support will be following during this bypass period.
- The amount that traffic from this customer will increase if rate limits are bypassed.

Process to Request a Bypass

1. Open a Production Engineering issue using the Rate limiting template and fill out the

applicable fields. Be sure to include:
   1. The IPs needed to be bypassed.
   1. A description of the rate limiting problem the customer is hitting and how it is effecting their systems (how severe is this for the customer).
   1. The amount of traffic increase we expect to see if the rate limit is bypassed.
   1. The plan of action the customer and/or Support will be following during this bypass period.
   1. The requested duration of the bypass.

The issue will be triaged by the Production Engineering::Foundations team. A request may result in questions for more information. If a request is deemed valid, the Foundations team will seek approval from Director+ level of infrastructure engineering. If approval is given, the Foundations team will work to schedule the bypass.

SECTION: engineering/infrastructure/rate-limiting/managing-limits.md

Overview

GitLab environments require a multi-layered approach to rate limiting to effectively protect services and resources.
Each layer provides distinct advantages and serves as a complementary control in a comprehensive defense in depth strategy to protect our platform.

Follow this guide when introducing or managing limits.

 Where to configure the limit

Use the following diagram to determine where the limit should be configured,
then follow the corresponding guidance for the appropriate configuration method.

mermaid
flowchart TD
    selfmanaged[Should GitLab.com and Self-Managed match?]

    ip-one[Is throttling by IP sufficient?]
    ip-two[Is throttling by IP sufficient?]

    dedicated[Should GitLab.com and Dedicated match?]

    subgraph layer[Configure the limit in...]
        subgraph app[Application]
            rackattack[RackAttack]
            appratelimiter[ApplicationRateLimiter]
        end
        subgraph cf[Cloudflare]
            cwm[cloudflare-waf-module rule]
            env[Environment-specific rule]
        end

```
    end

selfmanaged -- no --> ip-one
selfmanaged -- yes --> ip-two

ip-one -- yes --> dedicated
ip-one -- no --> appratelimiter

ip-two -- yes --> rackattack
ip-two -- no --> appratelimiter

dedicated -- yes --> cwm
dedicated -- no --> env
```

{{% alert title="Note" color="primary" %}}
This document is currently focused on inbound limits on HTTP traffic,
and may be expanded in the future to account for internal limits between services.
{{% /alert %}}

Considerations

Rate limits should be enabled by default. If this is not the case, then this process should be followed for the following cases:

- Introducing new rate limits
- Lowering existing rate limits
- Re-enabling disabled rate limits
- Increasing rate limits

Process

{{% alert title="Important" color="primary" %}}
In cases of incident remediation, see Rate Limiting Runbooks.
{{% /alert %}}

1. Determine if a rate limit already exists
   - Is there an Application limit for this already? What about Cloudflare?
   - Is it possible an existing limit could be adjusted (higher or lower)?
   - See Rate Limiting: Limits for where these limits are configured.
1. Compare proposed limit with existing limits
   - Will customers be negatively impacted by introducing this limit?
   - Are there risks to our platform if we don't introduce these limits?
1. Where possible, enable in log or track mode first
   - It should be left in this mode for at least one week to understand weekly traffic patterns.
   - This will allow you to gauge potential impact.
   - Use observability tooling (Cloudflare dashboard, logs, metrics) to analyze impact.
1. Produce evidence for the proposed limit
   - Why have you selected the value you have?

  - Using the findings from the previous step to support your proposal.
  - Are there any knock on effects to customers or backend systems we need to consider?
1. Determine a rollout plan
  - Will you use brownouts (temporarily introduce for short periods of time)?
  - If an Application Limit: will you use feature flags?
1. Inform relevant stakeholders of the change
  - Consider informing customersuccess, supportgitlab-com, and security.
  - Make sure they have access to documentation, and relevant information to help customers who contact them regarding the limit.
1. Communicate with customers
  - Announce the rate limits on the GitLab blog, see example for Projects, Groups, and Users APIs.
  - If an Application Limit: Document this change in the next release post.
  - Raise a contact request by following the Support: Contacting Customers workflow.
    - Notify of the change, and time frames where possible.
    - Provide guidance on any available remedies, workarounds, or best practices to help mitigate the impact.
1. Document the limits on docs.gitlab.com
  - Make sure the limit is documented, and if it's configurable, what the default is.
  - Note: This may not always be possible for Cloudflare limits.
1. Follow the Change Management process
  - Any change to rate limits is considered a Criticality 2 change, as they have the potential to disrupt traffic flow.
  - This requires approval from @gitlab-org/saas-platforms/inframanagers

Cloudflare

Enforcing limits at the edge network before traffic reaches the underlying GitLab infrastructure enables us to block malicious traffic before it consumes backend resources, protecting us against large-scale volumetric attacks. This is however limited in the configuration options we can use to limit on - primarily IP address, though there are a few other options.

1. Cloudflare rate limits are managed with Terraform, for GitLab.com through the cloudflare-waf-rules module. Rules added to this module will affect all other services using this module (currently GitLab Dedicated).
2. When creating a new rule, it is advised to instantiate it with action = "log" to analyze impact.
3. After ensuring the rate limit performs as expected, the rule can be set to action = "block" in Terraform.

GitLab Dedicated

Rate limits needed for only GitLab Dedicated Tenants (not GitLab.com) will need to be added to Terraform in Instrumentor. For further assistance please open an issue in the Dedicated Tracker or gdedicatedteam on Slack (internal link).

Application

Enforcing limits in the application level within GitLab itself enable us to be more

opinionated, as they are more context aware (understanding GitLab-specific resources) that provide us more granular control over specific features, and supports the ability to apply business logic and user/project-based dimensions to limiting decisions. New Application Rate Limits are configured in the GitLab application code, and should be introduced following the guide in Product Processes.

For information about rate limits that are currently configured within a GitLab instance, see the Rate Limits docs. For information about rate limits specifically configured to the GitLab.com instance, see Rate Limits on GitLab.com.

Rate limits in RackAttack

GitLab utilises RackAttack as middleware to throttle Rack requests. Most application-level rate limiting is managed with RackAttack. For instructions for configuring already existing rate limits, see the User and IP Rate Limits docs.

New rate limits can be configured by extending Gitlab::RackAttack and Gitlab::RackAttack::Request. Instructions for this can be found in the GitLab Development Docs.

{{% alert title="Important" color="primary" %}}
For new limits on GitLab.com, it is recommended to enable these in "Dry Run" (log) mode first. Instructions for this can be found in Runbooks.
{{% /alert %}}

You can read more information about rate limits specific to GitLab.com, alongside RackAttack configuration documentation in runbooks.

Rate limits in ApplicationRateLimiter

The GitLab application has simple rate limit logic that can be used to throttle certain actions which is used when we need more flexibility than what Rack Attack can provide, since it can throttle at the controller or API level. These rate limits are configured in applicationratelimiter.rb. The scope is up to the individual limit implementation and can be any ActiveRecord object or combination of multiple. It is commonly per-user or per-project (or both), but it can be anything, for example the RawController limits by project and path. Currently there is no way to bypass limits created in the ApplicationRateLimiter.

New rate limits may be created in the ApplicationRateLimiter by following the guide in GitLab docs.

Identifying Potentially Impacted Customers

There are several dimensions you can use when identifying impacted customers.

<table>
<tr>
<th>

Cloudflare
</th>

<td>

- IP
- Project ID (from URL)

</td>
</tr>
<tr>
<th>

RackAttack
</th>
<td>

- Username
- IP

</td>
</tr>
<tr>
<th>

ApplicationRateLimiter
</th>
<td>

- Username
- IP
- Project

</td>
</tr>
</table>

Monitoring and metrics around rate limits on GitLab.com are most easily accessed on the Rate Limiting Dashboard (internal link).

Using the Project ID to identify a customer namespace

If you have access to the project ID for requests identified as potentially hitting rate li